Cryptography and Network Security (BT14503) :

Data Encryption Standard

Dr. Shivangi Shukla

Assistant Professor

Computer Science and Engineering

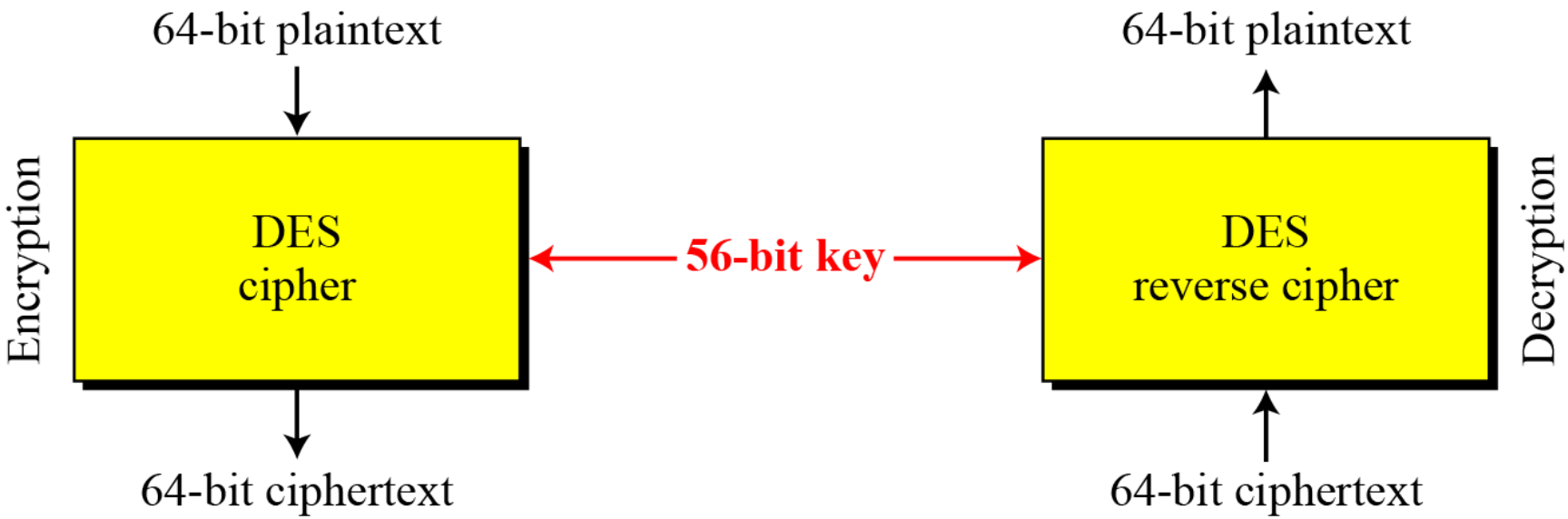
IIIT Pune



Introduction

- The Data Encryption Standard (DES) is a symmetric-key block cipher published by the National Institute of Standards and Technology (NIST).
- In 1973, NIST published a request for proposals for a national symmetric-key cryptosystem. A proposal from IBM, a modification of a project called Lucifer, was accepted as DES. DES was published in the Federal Register in March 1975 as a draft of the Federal Information Processing Standard (FIPS).

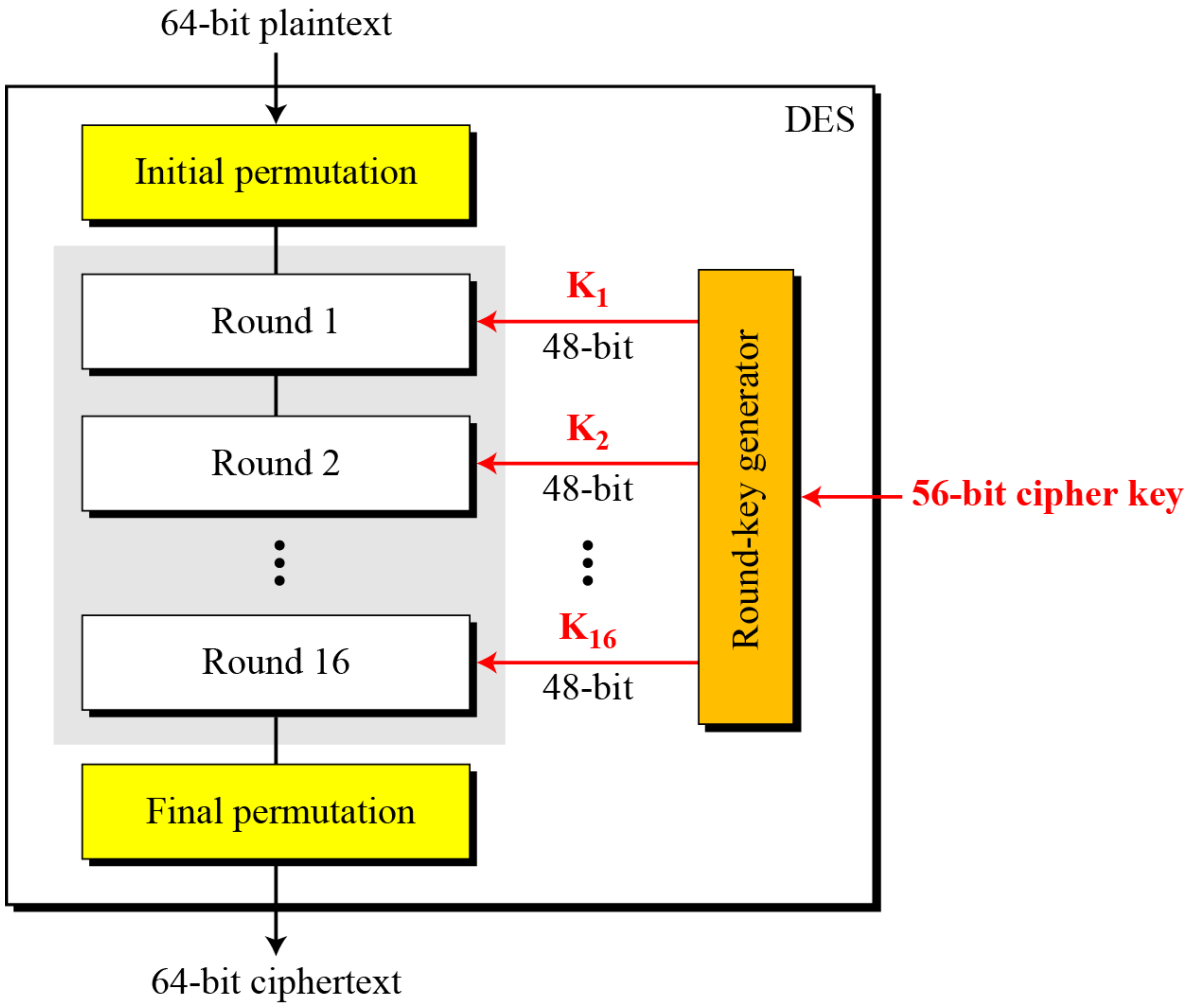
Overview


64-bit plaintext
Encryption
DES
cipher
64-bit ciphertext
56-bit key
64-bit plaintext
DES
reverse cipher
Decryption
64-bit ciphertext

Structure of DES

- The encryption process is made of
  - two permutations (P-boxes), which we call initial and final permutations, and
  - sixteen Feistel rounds
    - Each round uses a different 48-bit round key generated from the cipher key according to a predefined algorithm

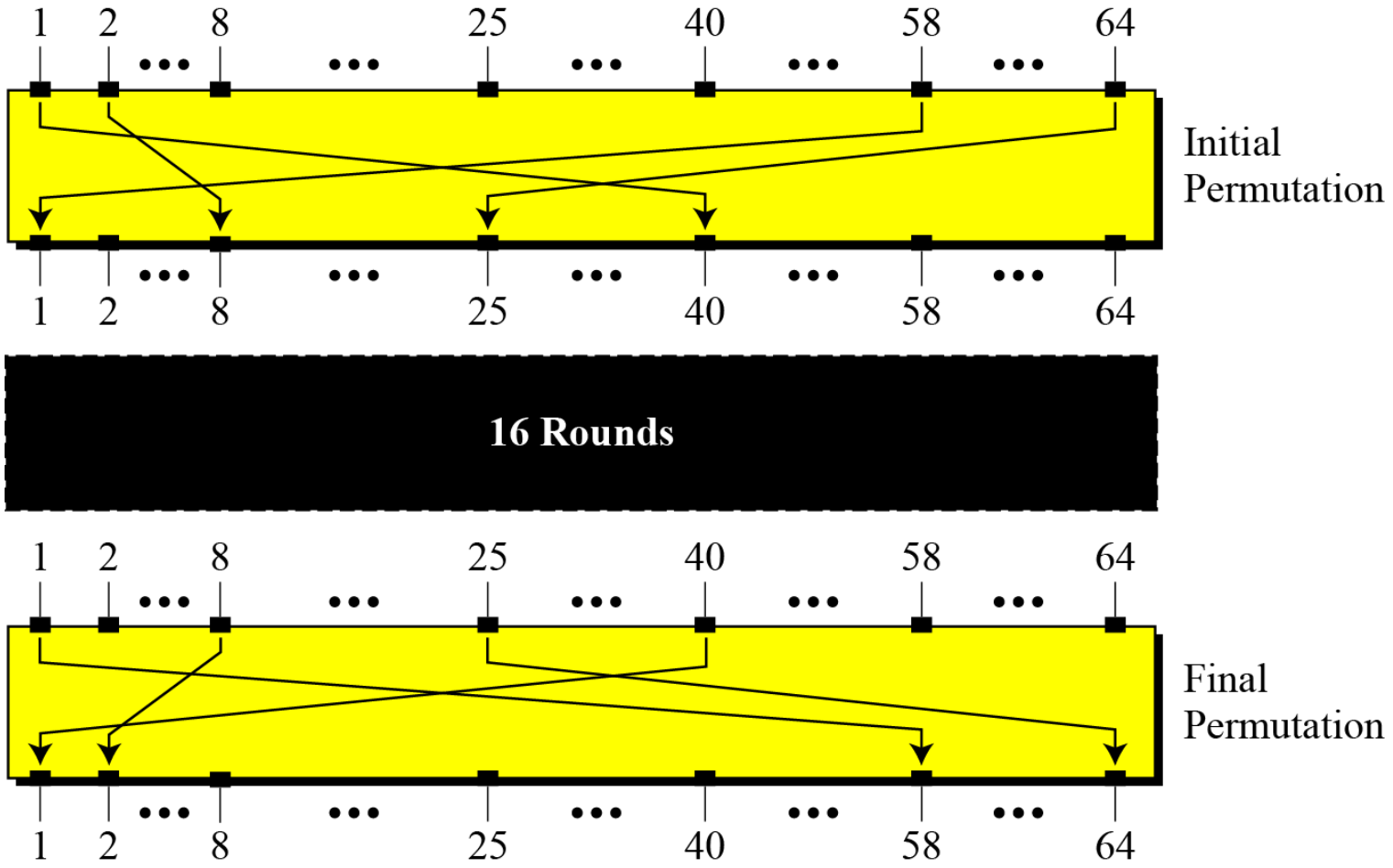
Structure of DES…



Initial and Final Permutations

- Each of these permutations takes a 64-bit input and permutes them according to a predefined rule.
- These permutations are keyless straight permutations that are the inverse of each other.

Initial and Final Permutations



Initial and Final permutations…

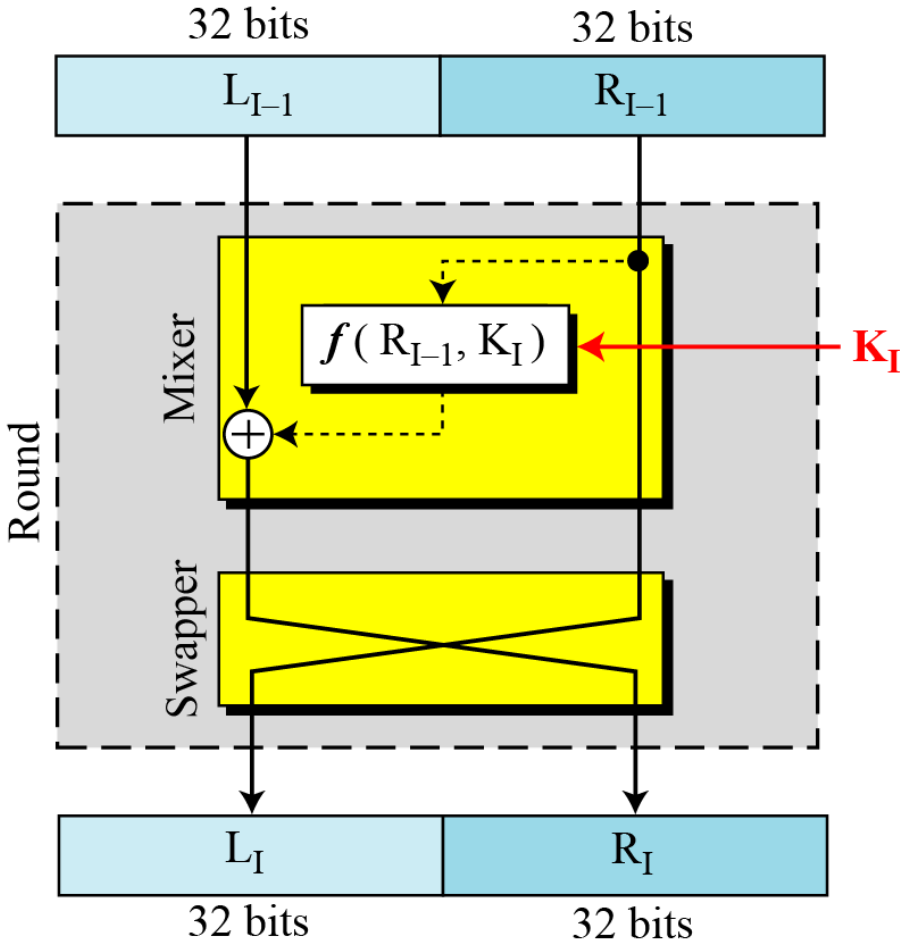
Initial Permutation								*Final Permutation*							
58	50	42	34	26	18	10	02	40	08	48	16	56	24	64	32
60	52	44	36	28	20	12	04	39	07	47	15	55	23	63	31
62	54	46	38	30	22	14	06	38	06	46	14	54	22	62	30
64	56	48	40	32	24	16	08	37	05	45	13	53	21	61	29
57	49	41	33	25	17	09	01	36	04	44	12	52	20	60	28
59	51	43	35	27	19	11	03	35	03	43	11	51	19	59	27
61	53	45	37	29	21	13	05	34	02	42	10	50	18	58	26
63	55	47	39	31	23	15	07	33	01	41	09	49	17	57	25

Initial and Final Permutations

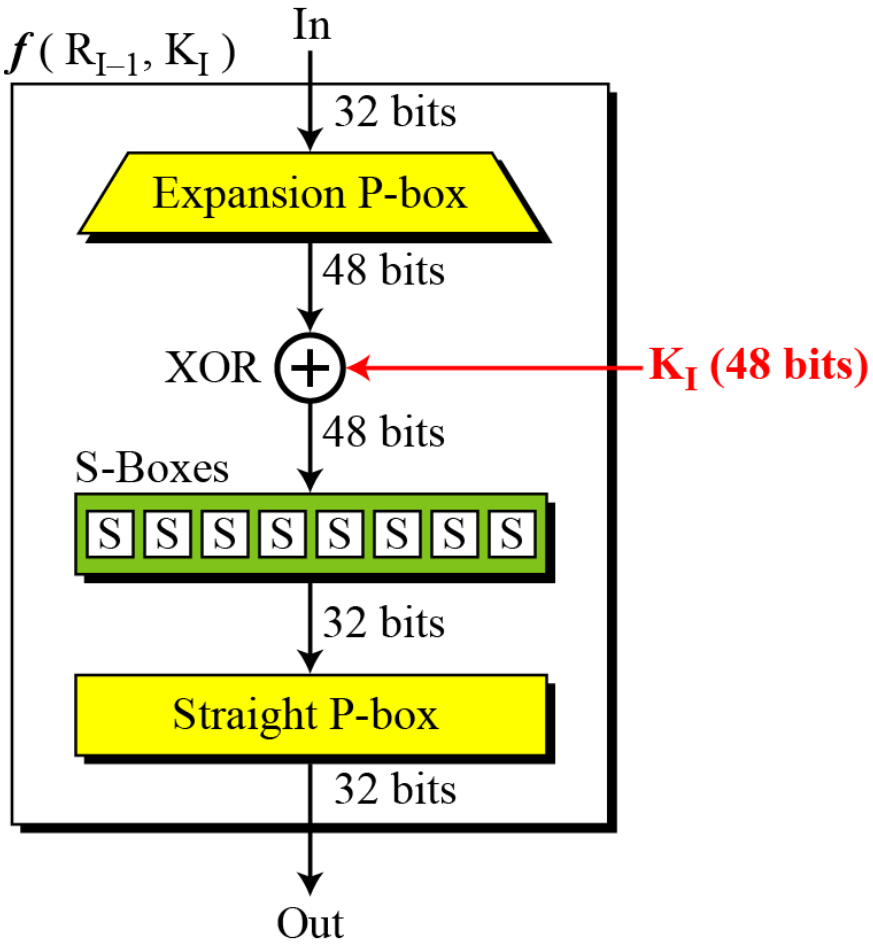
- These two permutations have no cryptography significance in DES.
- Both permutations are keyless and predetermined.
- The reason they are included in DES is not clear and has not been revealed by the DES designers.
- The guess is that DES was designed to be implemented in hardware (on chips) and that these two complex permutations may prevent a software simulation of the mechanism

Rounds

- 16 rounds of Feistel cipher

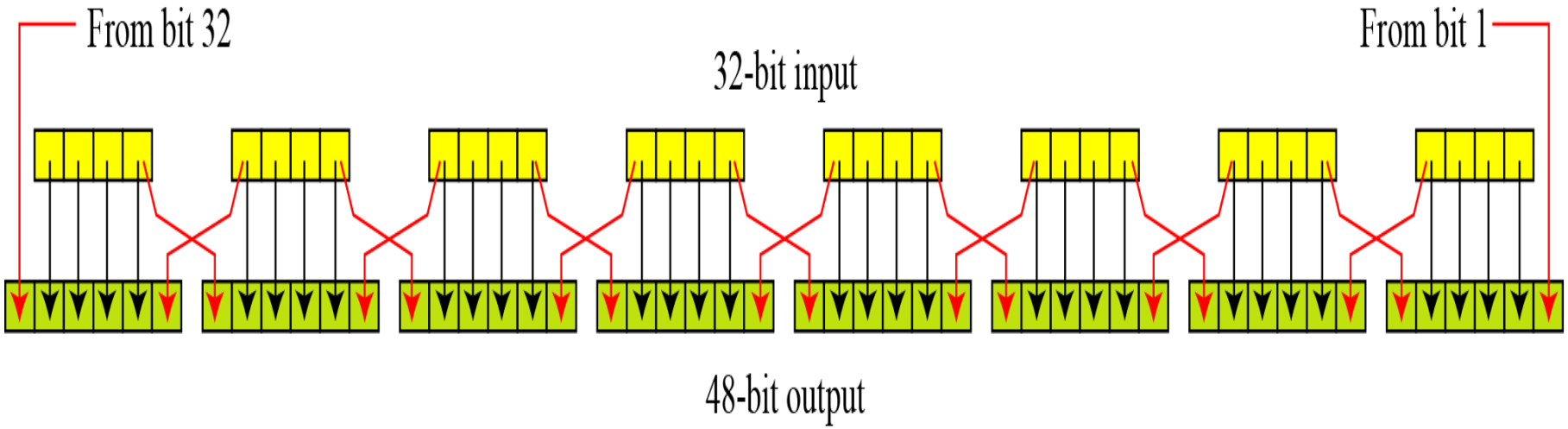


DES function



DES function…

- Expansion P-box
  - Since R_{l-1} is a 32-bit input and K_l is a 48-bit key, we first need to expand R_{l-1} to 48 bits.



DES function…

- Expansion P-box
  - R_{l-1} is a 32-bit input and K_l is a 48-bit key, we first need to expand R_{l-1} to 48 bits.
  - R_{l-1} is divided into 8 4-bit sections. Each 4-bit section is then expanded to 6 bits.
  - This expansion permutation follows a predetermined rule.
  - For each section, input bits 1, 2, 3, and 4 are copied to output bits 2, 3, 4, and 5, respectively.
  - Output bit 1 comes from bit 4 of the previous section; output bit 6 comes from bit 1 of the next section.
  - If sections 1 and 8 can be considered adjacent sections, the same rule applies to bits 1 and 32.

DES function…

- Expansion P-box
  - Since R_{l-1} is a 32-bit input and K_l is a 48-bit key, we first need to expand R_{l-1} to 48 bits.

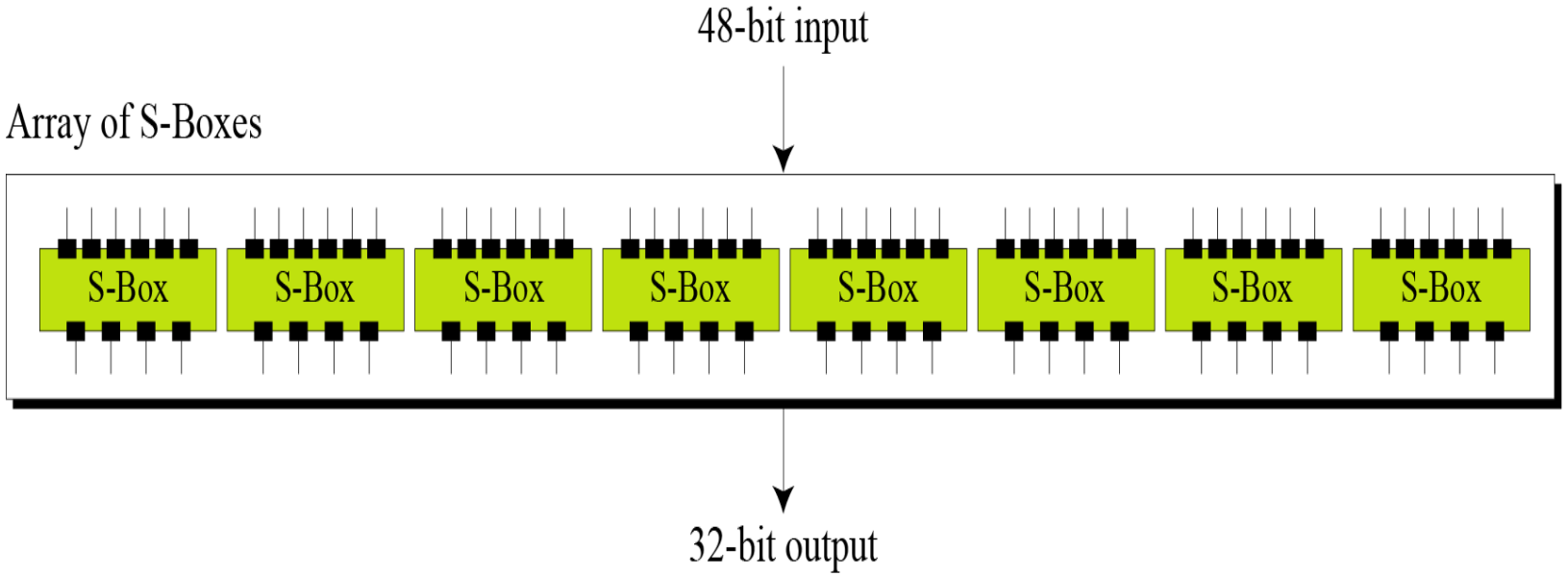
32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

DES function…

- Whitener
  - After the expansion permutation, DES uses the XOR operation on the expanded right section and the round key.
    - $f(R_{I-1}, K_I) \oplus K_I$
  - Note that both the right section and the key are 48-bits in length.
  - Also note that the round key is used only in this operation.

DES function…

- S-Boxes
  - used for confusion
  - DES uses 8 S-boxes, each with 6-bit input and 4-bit output

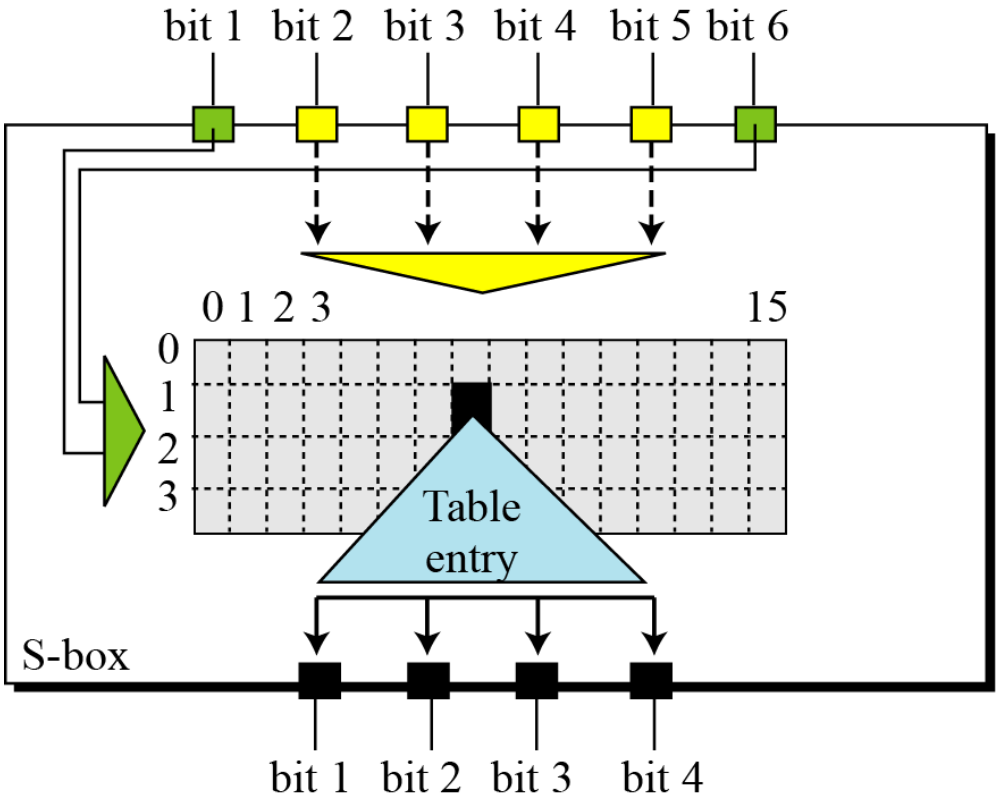


DES function…

- S-Boxes
  - The substitution in each box follows a pre-determined rule based on a 4-row by 16-column table.
  - The combination of bits 1 and 6 of the input defines one of four rows; the combination of bits 2 through 5 defines one of the sixteen columns

DES function…

- S-Boxes



DES function…

- Permutations for S-Box-1

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

- Permutations for S-Box-2

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	15	01	08	14	06	11	03	04	09	07	02	13	12	00	05	10
1	03	13	04	07	15	02	08	14	12	00	01	10	06	09	11	05
2	00	14	07	11	10	04	13	01	05	08	12	06	09	03	02	15
3	13	08	10	01	03	15	04	02	11	06	07	12	00	05	14	09

DES function…

- Permutations for S-Box-3

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	10	00	09	14	06	03	15	05	01	13	12	07	11	04	02	08
1	13	07	00	09	03	04	06	10	02	08	05	14	12	11	15	01
2	13	06	04	09	08	15	03	00	11	01	02	12	05	10	14	07
3	01	10	13	00	06	09	08	07	04	15	14	03	11	05	02	12

- Permutations for S-Box-4

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	07	13	14	03	00	6	09	10	1	02	08	05	11	12	04	15
1	13	08	11	05	06	15	00	03	04	07	02	12	01	10	14	09
2	10	06	09	00	12	11	07	13	15	01	03	14	05	02	08	04
3	03	15	00	06	10	01	13	08	09	04	05	11	12	07	02	14

DES function…

- Permutations for S-Box-5

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	02	12	04	01	07	10	11	06	08	05	03	15	13	00	14	09
1	14	11	02	12	04	07	13	01	05	00	15	10	03	09	08	06
2	04	02	01	11	10	13	07	08	15	09	12	05	06	03	00	14
3	11	08	12	07	01	14	02	13	06	15	00	09	10	04	05	03

- Permutations for S-Box-6

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	12	01	10	15	09	02	06	08	00	13	03	04	14	07	05	11
1	10	15	04	02	07	12	09	05	06	01	13	14	00	11	03	08
2	09	14	15	05	02	08	12	03	07	00	04	10	01	13	11	06
3	04	03	02	12	09	05	15	10	11	14	01	07	10	00	08	13

DES function…

- Permutations for S-Box-7

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	4	11	2	14	15	00	08	13	03	12	09	07	05	10	06	01
1	13	00	11	07	04	09	01	10	14	03	05	12	02	15	08	06
2	01	04	11	13	12	03	07	14	10	15	06	08	00	05	09	02
3	06	11	13	08	01	04	10	07	09	05	00	15	14	02	03	12

- Permutations for S-box 8

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
0	13	02	08	04	06	15	11	01	10	09	03	14	05	00	12	07
1	01	15	13	08	10	03	07	04	12	05	06	11	10	14	09	02
2	07	11	04	01	09	12	14	02	00	06	10	10	15	03	05	08
3	02	01	14	07	04	10	8	13	15	12	09	09	03	05	06	11

DES function…

- Straight P-Box

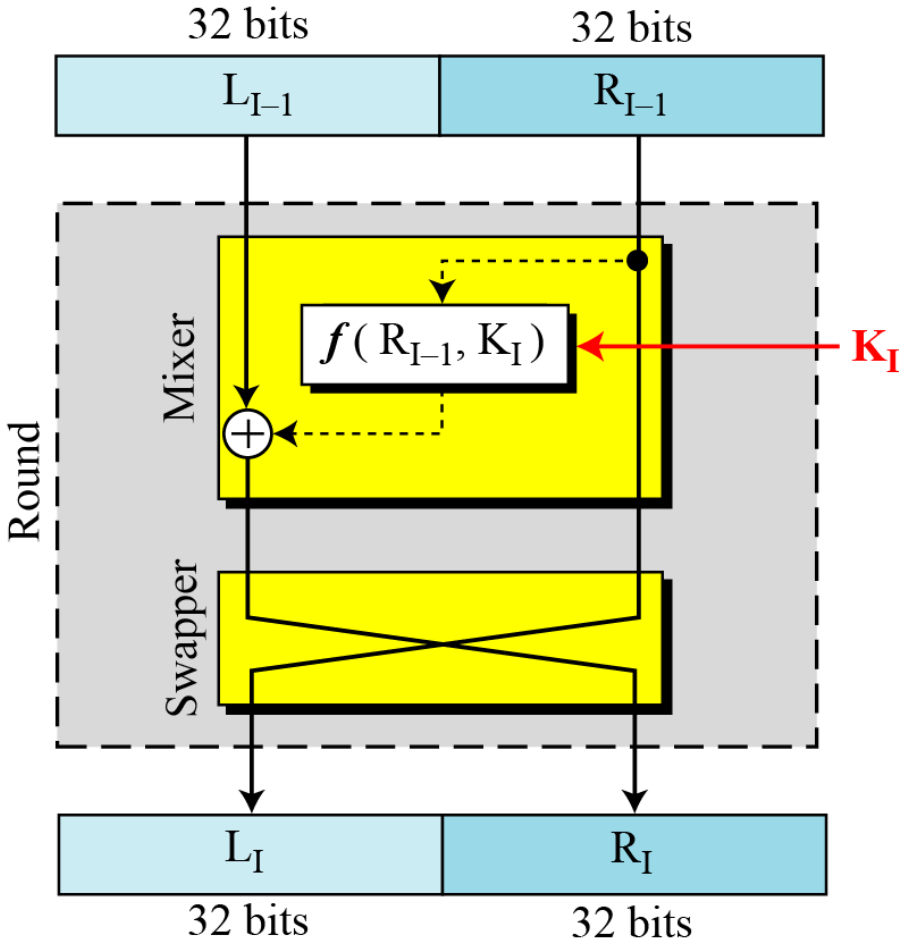
16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Cipher and reverse cipher

- Using mixers and swappers, we can create the cipher and reverse cipher, each having 16 rounds.
- One Approach
  - To achieve this goal, one approach is to make the last round (round 16) different from the others; it has only a mixer and no swapper.

Cipher and reverse cipher…

- 16 rounds of Feistel cipher



Cipher and reverse cipher...

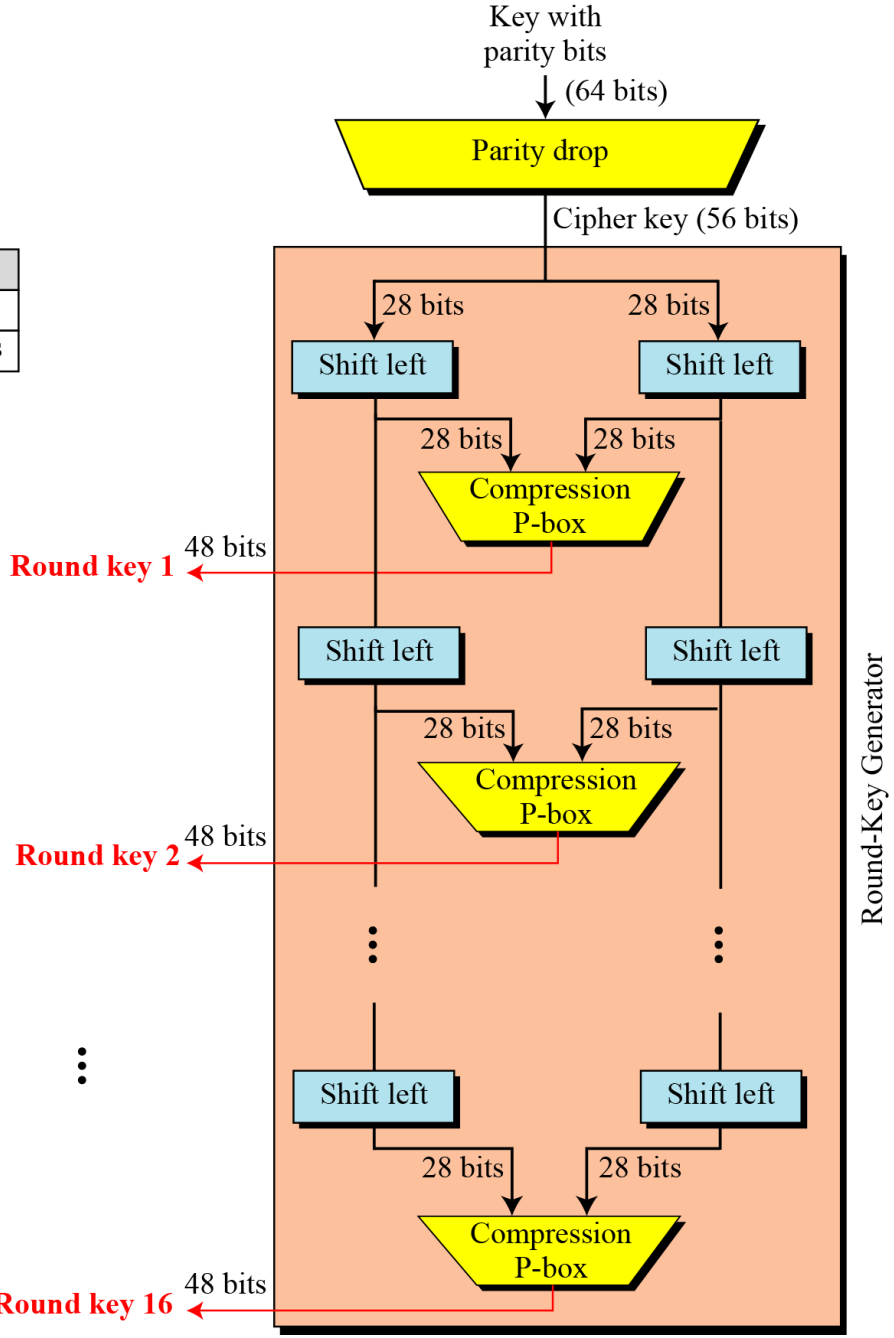
- Alternative approach
  - We can make all 16 rounds the same by including one swapper to the 16th round and add an extra swapper after that.
    - The two swappers cancel effect of each other

Key Generation

- The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key.

Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits



Key Generation

- Parity Drop
  - It drop parity bits (bits 8, 16, 24, 32.....64) from 64-bit key
    - and permutes rest of them as per Parity bit drop table

Parity bit drop table

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	38
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Key Generation

- Shift Left
  - Key is divided into two 28-bit parts
  - Each part is shifted left (circular shift) one or two bits
  - In rounds 1, 2, 9 and 16 shifting is one bit and
    - other rounds it is shifted by two bits

Number of shift bits

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Key Generation

- Compression P-box
  - changes 56 bits to 48-bits which is used as key for a round

Key compression table

14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

DES: an example of encipherment

Plaintext: 123456ABCD132536 Key: AABB09182736CCDD
CipherText: C0B7A8D05F3A829C

Plaintext: 123456ABCD132536			
*After initial permutation:*14A7D67818CA18AD After splitting: L_0=14A7D678 R_0=18CA18AD			
Round	*Left*	*Right*	*Round Key*
Round 1	18CA18AD	5A78E394	194CD072DE8C
Round 2	5A78E394	4A1210F6	4568581ABCCE
Round 3	4A1210F6	B8089591	06EDA4ACF5B5
Round 4	B8089591	236779C2	DA2D032B6EE3

DES: an example of encipherment...

Round 5	236779C2	A15A4B87	69A629FEC913
Round 6	A15A4B87	2E8F9C65	C1948E87475E
Round 7	2E8F9C65	A9FC20A3	708AD2DDB3C0
Round 8	A9FC20A3	308BEE97	34F822F0C66D
Round 9	308BEE97	10AF9D37	84BB4473DCCC
Round 10	10AF9D37	6CA6CB20	02765708B5BF
Round 11	6CA6CB20	FF3C485F	6D5560AF7CA5
Round 12	FF3C485F	22A5963B	C2C1E96A4BF3
Round 13	22A5963B	387CCDAA	99C31397C91F
Round 14	387CCDAA	BD2DD2AB	251B8BC717D0
Round 15	BD2DD2AB	CF26B472	3330C5D9A36D
Round 16	19BA9212	CF26B472	181C5D75C66D
After combination: 19BA9212CF26B472			
Ciphertext: C0B7A8D05F3A829C			*(after final permutation)*

DES: an example of decipherment

Ciphertext: C0B7A8D05F3A829C			
After initial permutation: 19BA9212CF26B472 After splitting: L_0=19BA9212 R_0=CF26B472			
Round	*Left*	*Right*	*Round Key*
Round 1	CF26B472	BD2DD2AB	181C5D75C66D
Round 2	BD2DD2AB	387CCDAA	3330C5D9A36D
. . .	. . .	. . .	. . .
Round 15	5A78E394	18CA18AD	4568581ABCCE
Round 16	14A7D678	18CA18AD	194CD072DE8C
After combination: 14A7D67818CA18AD			
Plaintext:123456ABCD132536			(after final permutation)

DES: Analysis

- Two desirable properties
  - Avalanche effect
  - Completeness

Avalanche effect

- To check the avalanche effect in DES, let us encrypt two plaintext blocks (with the same key) that differ only in one bit and observe the differences in the number of bits in each round.

Plaintext: 0000000000000000	Key: 22234512987ABB23
Ciphertext: 4789FD476E82A5F1	
Plaintext: 0000000000000001	Key: 22234512987ABB23
Ciphertext: 0A4ED5C15A63FEA3	

Avalanche effect...

- Ciphertext blocks differ in 29 bits.
  - i.e. changing approximately 1.5 percent of the plaintext creates a change of approximately 45 percent in the ciphertext.

Rounds	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*	*16*
Bit differences	1	6	20	29	30	33	32	29	32	39	33	28	30	31	30	29

Completeness

- Completeness effect means that each bit of the ciphertext needs to depend on many bits on the plaintext.
  - S-Box
  - P-Box
  - 16 rounds of Feistel blocks

DES Weaknesses

- Weakness in key
  - Weak Keys: Four out of 2^{56} possible keys are called weak keys.
  - A weak key is the one that, after parity drop operation, consists either of all 0s, all 1s, or half 0s and half 1s.

Table 6.18 *Weak keys*

Keys before parities drop (64 bits)	*Actual key (56 bits)*
0101 0101 0101 0101	0000000 0000000
1F1F 1F1F 0E0E 0E0E	0000000 FFFFFFF
E0E0 E0E0 F1F1 F1F1	FFFFFFF 0000000
FEFE FEFE FEFE FEFE	FFFFFFF FFFFFFF

DES Weaknesses..

- The round keys created from any of these weak keys are the **same** and have the same pattern as the cipher key.
  - For example, the sixteen round keys created from the first key is all made of 0s; the one from the second is made of half 0s and half 1s.
- The reason is that the key-generation algorithm first divides the cipher key into two halves.
  - Shifting or permutation of a block does not change the block if it is made of all 0s or all 1s.

DES Weaknesses..

- The problem with weak keys is:
  - Encrypt a block with weak key and then encrypt the result with same weak key again
    - We get the original block
  - In other works, $E_k(E_k(P)) = P$

DES Weaknesses..

- Semi-Weak Keys
  - There are six key pairs that are called semi-weak keys.
  - **A semi-weak** key that has a special relationship with another key in pair. For a pair of semi-weak keys K_1 and K_2

$$E_{K_1}\left(E_{K_2}(P)\right) = \mathrm{P}$$

DES Weaknesses..

First key in the pair	*Second key in the pair*
01FE 01FE 01FE 01FE	FE01 FE01 FE01 FE01
1FE0 1FE0 0EF1 0EF1	E01F E01F F10E F10E
01E0 01E1 01F1 01F1	E001 E001 F101 F101
1FFE 1FFE 0EFE 0EFE	FE1F FE1F FE0E FE0E
011F 011F 010E 010E	1F01 1F01 0E01 0E01
E0FE E0FE F1FE F1FE	FEE0 FEE0 FEF1 FEF1

Semi-weak Key Pairs

DES Weaknesses..

- Possible Weak Keys:
  - There are also 48 keys that are called possible weak keys.
  - A possible weak key is a key that creates only four distinct round keys;
    - in other words, the sixteen round keys are divided into four groups and each group is made of four equal round keys.

DES Weaknesses..

- Key Complement:
  - In the key domain (2^{56}),
    - definitely half of the keys are complement of the other half.
    - A key complement can be made by inverting (changing 0 to 1 or 1 to 0) each bit in the key.
    - If we encrypt the complement of plaintext with the complement of the key, we get the complement of the ciphertext.
    - Eve does not have to test all 2^{56} possible keys, she can test only half of them and then complement the result.

Security of DES

- Brute-Force Attack
  - DES can be broken using 2^{55} encryptions.
  - However, today most applications use either 3DES with two keys (key size of 112) or 3DES with three keys (key size of 168).
  - These two multiple-DES versions make DES resistant to brute-force attacks

Security of DES..

- Differential Cryptanalysis
  - It has been shown that DES can be broken using differential cryptanalysis, if we have 2^{47} chosen plaintexts or 2^{55} known plaintexts.
  - Although this looks more efficient than a brute-force attack, finding 2^{47} chosen plaintexts or 2^{55} know plaintexts is impractical. Therefore, we can say that DES is resistant to differential cryptanalysis.

Security of DES..

- Linear Cryptanalysis
  - DES is more vulnerable to linear cryptanalysis than to differential cryptanalysis,
    - S-boxes are not very resistant to linear cryptanalysis.
  - It has been shown that DES can be broken using 2^{43} pairs of known plaintexts.

Multiple DES

- With available technology and the possibility of parallel processing, a brute-force attack on DES is feasible.
- One solution is to use multiple (cascaded) instances of DES with multiple keys

Multiple DES..

- A substitution that maps every possible input to every possible output is a group, with the mappings as the set elements and the composition as the operator.
- In this case, using two consecutive mappings is useless because we can always find the third mapping that is equivalent to the composition of the two (closure property).
- This means that if DES is a group, using double DES with two keys k_1 and k_2 is useless because a single DES with key k_3 does the same thing
- It should be noted that DES does not form a group under composition

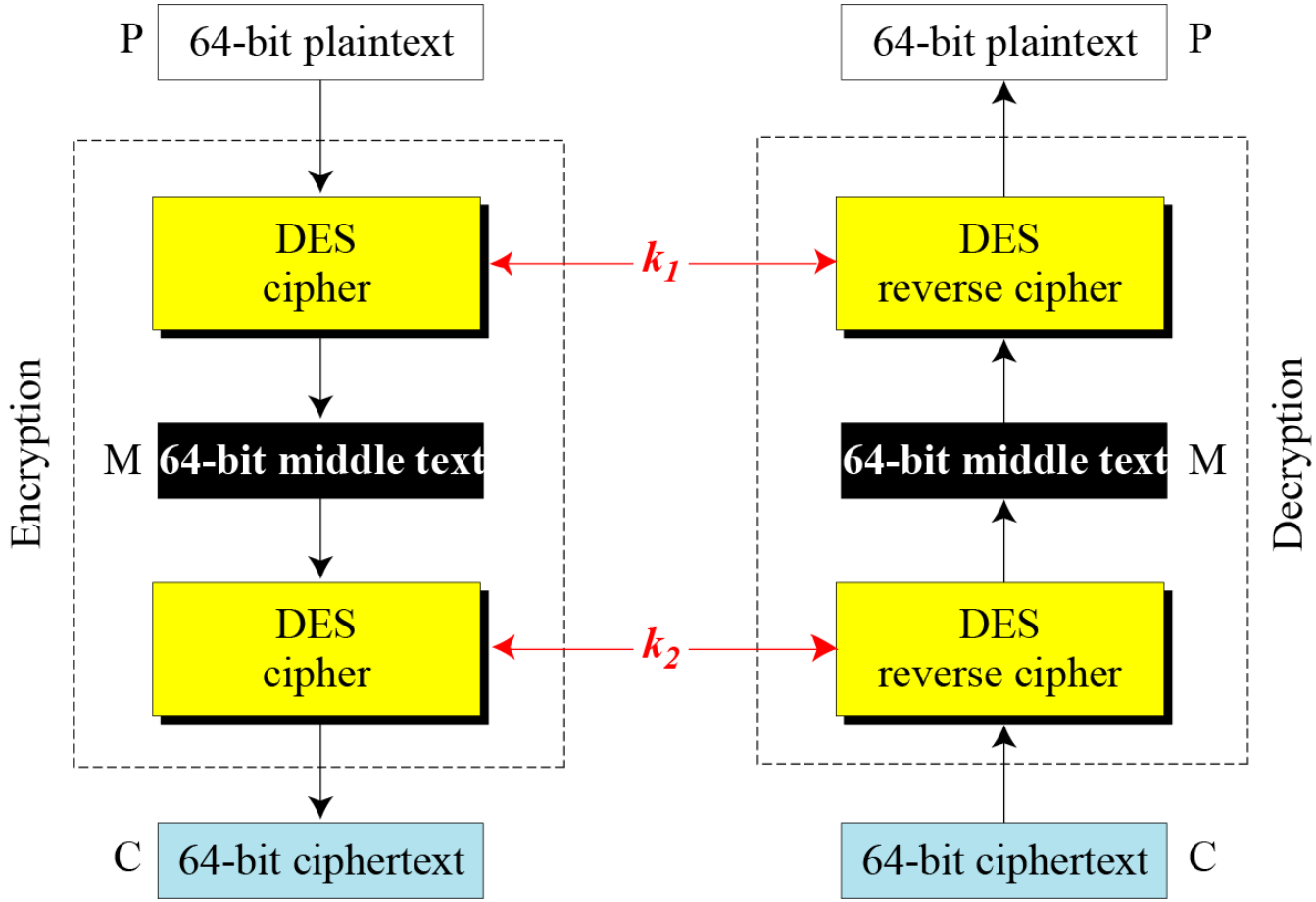
Double DES

- The first approach is to use double DES (2DES).
- In this approach, we use two instances of DES ciphers for encryption and two instances of reverse ciphers for decryption.
- Each instance uses a different key, which means that the size of the key is now doubled (112 bits).

Meet-in-the-Middle Attack

- It looks like double DES increases the number of tests for key search from 2^{56} (in single DES) to 2^{112} (in double DES).
- However, using a known-plaintext attack called meet-in-the-middle attack proves that double DES improves this vulnerability slightly (to 2^{57} tests), but not tremendously (to 2^{112}).

Meet-in-the-Middle Attack..



$$M = E_{k_1}(P) \quad \text{and} \quad M = D_{k_2}(C)$$

Meet-in-the-Middle Attack..

- Assume that Eve has intercepted a previous pair P and C (known-plaintext attack).
- Based on the first relation, Eve encrypts P using all possible values (2^{56}) of k_1 and records all values obtained for M.
- Based on the second relation, Eve decrypts C using all possible values (2^{56}) of k_2 and records all values obtained for M.
- Eve creates two tables sorted by M values and then compares M until she finds those pairs of k_1 and k_1 for which the value of M is the same
- Note that there must be at least one pair because she is doing exhaustive search on the combination of two keys

Meet-in-the-Middle Attack..

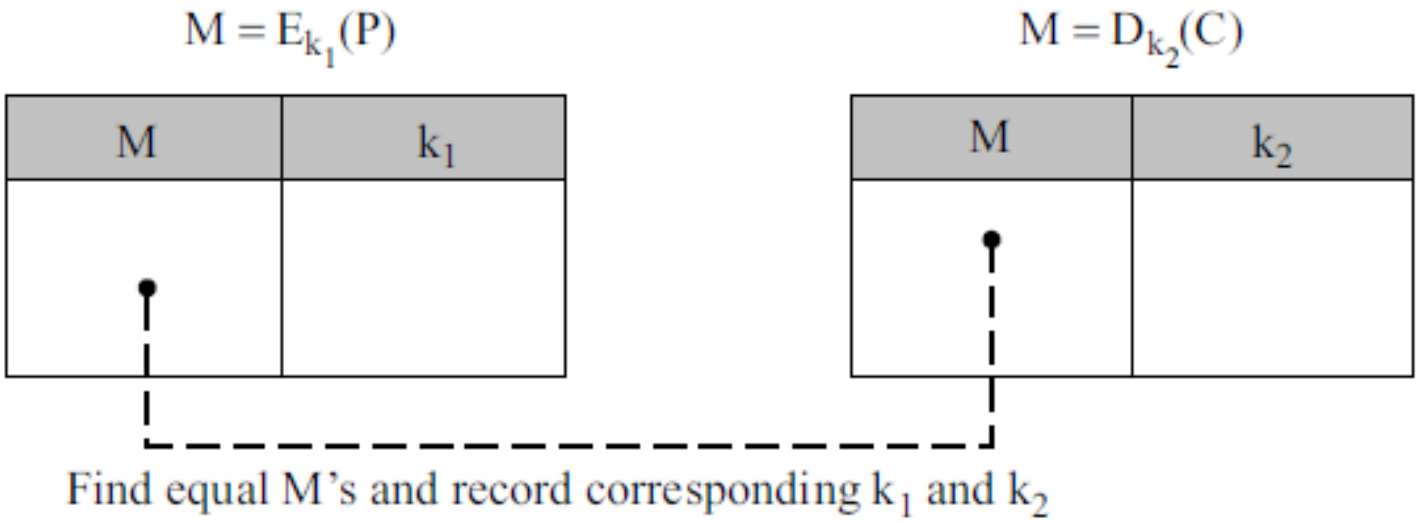


Fig 1: Table for meet-in-the-middle attack

- This means that instead of using 2^{112} key-search tests, Eve uses 2^{56} key-search tests two times
- Thus, moving from single DES to double DES, we have increased the strength from 2^{56} to 2^{57}

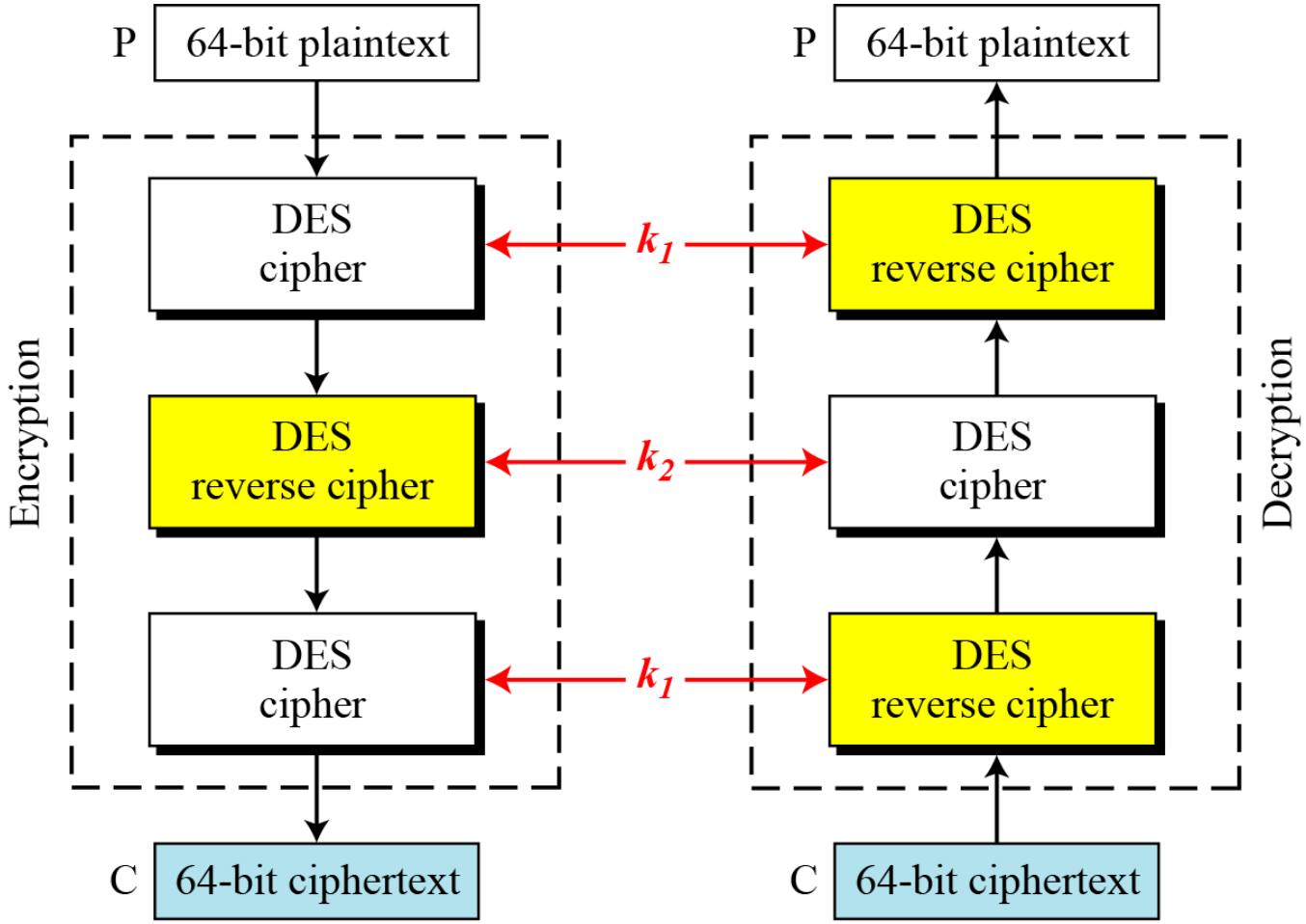
Triple DES

- To improve the security of DES, triple DES (3DES) was proposed.
- This uses three stages of DES for encryption and decryption.
- Two versions of triple DES are in use today:
  - triple DES with two keys and
  - triple DES with three keys

Triple DES..

- In triple DES with two keys, there are only two keys: k_1 and k_2.
- The first and the third stages use k_1; the second stage uses k_2.
- To make triple DES compatible with single DES,
  - the middle stage uses decryption (reverse cipher) in the encryption site
  - and encryption (cipher) in the decryption site.

Triple DES..


P
64-bit plaintext
DES cipher
DES reverse cipher
DES cipher
C
64-bit ciphertext
Encryption
k_1
k_2
k_1
P
64-bit plaintext
DES reverse cipher
DES cipher
DES reverse cipher
C
64-bit ciphertext
Decryption

Triple DES..

- In triple DES with three keys, there are keys: k_1, k_2 and k_3.

